Reference Architecture:

Opportunity in Lab Management

Version 1.0

Feng Xia

January 12, 2021

DRAFT COPY

Abstract

Managing a lab that functions as an IT infrastructure should be treated as managing a data center. We need rules to protect its integrity, tools to collect information and apply changes, check and balance to monitor its status and remove policy breach.

Contents

1 Challenges & misconceptions **3**

2 proposal **5**

3 function maturity levels **6**

4 function domains **7**

5 where we are today **8**

5.1 inventory 8

5.1.1 manually 8

5.1.2 semi-automatic 8

5.1.3 discovery 9

5.2 transportation 9

5.3 info query/parse/save 9

5.4 report 10

5.5 apply action 11

6 TODOs **12**

6.1 policies 12

6.1.1 TODO rebase customization to upstream version 12

6.1.2 TODO backup and restore TOR running configs 12

6.1.3 TODO achieve lab level isolation 12

6.1.4 TODO achieve rack level isolation 13

6.1.5 TODO switch user management 13

6.1.6 TODO BMC user management 13

6.2 inventory 13

6.2.1 TODO unify switch name 13

6.2.2 TODO unify BMC name 13

6.2.3 TODO re-examine hardware inventory, manually 13
6.2.4 TODO new web form to relocate device easily 14
6.2.5 TODO new web form to assign IP to a device 14
6.2.6 TODO re-examine and re-org stack grouping 14
6.3 info query/parse/save 14
6.3.1 TODO verify physical connection algorithm 14
6.3.2 TODO scale build inventory & meta from (ip,user,pwd) 15
6.3.3 TODO develop scan of storage controller 15
6.3.4 TODO develop scan of Pica 15
6.3.5 TODO develop scan of DHCP service 15
6.3.6 TODO research detection of VM vs. BM 15
6.4 report 16
6.4.1 TODO build report given a IP range 16
6.4.2 TODO build report of a stack 16
6.4.3 TODO build report of a grouping 16
6.4.4 TODO build report of physical connections 16

7 Opportunity **16**

8 References **17**

List of Tables

1 transportation 9
2 info query/parse/save 9
3 report 11
4 actions 11

List of Figures

1 Five levels of capabilities 7

Lab management needs to be hardened. When a problem arises at lab level, be it a network issue, a rougue DHCP, or IT demanding security patches, the lab will inevitably be put in a mode of emergency. Stake is high, not only in term of potential damage, but in suspending development, thus putting progress on hold.

The second problem is the lack of source of truth. We have information in Confluence, in Excel, sometimes in a person's head only. We make the best effort to maintain these information and hope they are up to date. Hoever, when operating under pressure, too often we found them lacking, conflicting, and outright missing.

Thirdly, we need a more efficient way to handle these fires and fire drills than throwing bodies at it. Often these episodes quickly turned into an *all hands on deck* scenario. It's not an issue by itself. But w/ the deficiency of source of truth, it is difficult to coordinate while maintain a full view of coverage. Thus, the quality of actions went without check. A best effort is not adequate in face of its objective.

Fourth, we don't have capacity to implement prevention measure. This has many factors. It's a cliche to talk about the benefits of managing proactively vs. passively. In order to be proactive, we need two things in place: rules, and means to enforce them. Commonly this is an area of enthusiasm that people's ideas become SOP, written, endorsed, and executed. But before diving into it, we must consider the practical side of our challenges.

1 Challenges & misconceptions

1. **sheer ratio problem**: Ricky is the only one managing RTP lab equipments. I imagine similar ratio will be found in Romania and India. If counting dev to Ricky

ratio, it's easily 10:1 or higher; if counting racks to Ricky ratio, it's in the 20:1; and if counting equipments to Ricky ratio, it's in the 100s:1. He is overwhelmed, and he should be.

2. **multi-tenants**: Our lab is shared. We have equipments owned by other DCG teams that reside physically in our rack, share our network, can break *our* rules, but without accountability nor our control. The only course of action has been to shut down these equipments in those drills. This, I think both we and owner would agree, isn't ideal.

   Further, within our own team, we have multiple development streams and each owns certain number of stacks and so on. They have their own networking requirement, deployment, development driven by *innovation*. Where do we strike a balance between flexibility and the rules?

3. **variety of hardware & software**: We have inherited and collected a mix bag of hardware. Not only the models are non-homogeneous, but the firmware have quirks because some are half-baked lab version, but a released one. Some can be upgraded; others can not. This adds the complexity of knowledge we have to mantain as long as this variety is in service. Documenting them isn't enough. Imagine in face of a fire, who is going to read through the *fine prints* in a manual in order to operate the water hose properly?

4. **catch-22**: When we break company policy, lab gets disconnected from campus. All the remote access are gone. This makes diagnosing the issue harder when we mostly need it than in normal time. In all cases we had before, the root cause was always a software change by a tenant, such as an unwanted DHCP service or a new switch config. They can be reverted, usually easily, by the person who has that knowledge, provides that s/he has the accss, chicken & egg.

5. **reversed authority pyramid**: We have treated our lab and its management as the bottom rung of our authority chain. This is reversed. Maintaining the common infrastructure that floats all our intellectual properties should be at the top of our power chain. Lab management should the captain on a ship — what s/he says is the law.

6. **as a development project**: Lab management requires knowledge and discipline no less than other projects. We have underestimated its complexity. It's a misconception that lab is about stacking up metal boxes and plugging in some

DRAFT COPY

wires. It's far more than that. It is really a hub of knoweledge where all of our teams' come to one place, and are compatible to each other. We should treat it as a mockup data center in which we are hoping to deploy our product one day. Thus, it requires the level of all factors in managing a datacenter. Some may be scaled down due to our footprints. But the basics remain the same and can't be brushed aside.

2 proposal

> Use Netbox as the source of truth of our lab.

Netbox is a tool developed by Digital Ocean, the 3rd largest hosting company in the world. It's also one of the eleven services of LOC-A, and is the core competence of LOC-A's automation.

Netbox has built-in models for all critical aspects of a data center. We and LOC-A have since extended models and added functions catering to our lab use.

We have also considered confluent and xClarity, but found them either too CLI centric, thus limiting the capability of reporting after data are collected, or lack of data models for information we want to manage. Considering the challenges discussed, it's advised to use a code w/ 100% control within our team so we can adapt it quickly. This consideration, therefore, marks these two tools less appealing than Netbox as the core. However, we are not to dismiss the power of these tools at all. Netbox's architecture will work nicely with them for data collection or command execution, especially on Lenovo hardware.

We have also considered Cloudform, which Lenovo has developed xClarity plugin. It can provide a single pane of glass. But itself is manager of managers, whereas we need a first line *manager* tool. Further, until we have established some low level management, using Cloudform provides limited benefit because it has no one to talk to.

Last, we have also considered deploying OSP or K8S or CP as a super platform. This certainly unifies some development variations such as VMs, and can potentially simplifies networking management. But considering the nature of our team and projects are often infrastructure centric, inevitably baremetals are essential building blocks. Therefore, managing baremetal properly outside these projects is still a must-have.

DRAFT COPY

3 function maturity levels

Maturity level indicates how powerful, or intelligent, the tool is. The higher the level, the more *intelligent* it is, and of course, the harder to achieve. In review, we follow the 5-phase maturity model I have in other place as our framework.

1. **inventory**: specify target, eg. HW list, IP list. The minimal, the better.
2. **reality/expectation**: read/query reality info of target, eg. machine model, serial number. In order for the two being comparable, it is critical to realize that the two share the *exact same* data models.
3. **diff**: where we define rules & policies, thus leading to identify rule offenders.
4. **design reality**: day-0 function when the design is ahead of reality. In other words, we enforce a design to become a reality, eg. re-config a VLAN.
5. **diff &rarr**; design: day-1+ function where we are expecting the system to sustain the design by itself, thus auto-correcting if it drifts from design. As you can see, only when it has the capability to apply a design could it now influence a reality.

DRAFT COPY

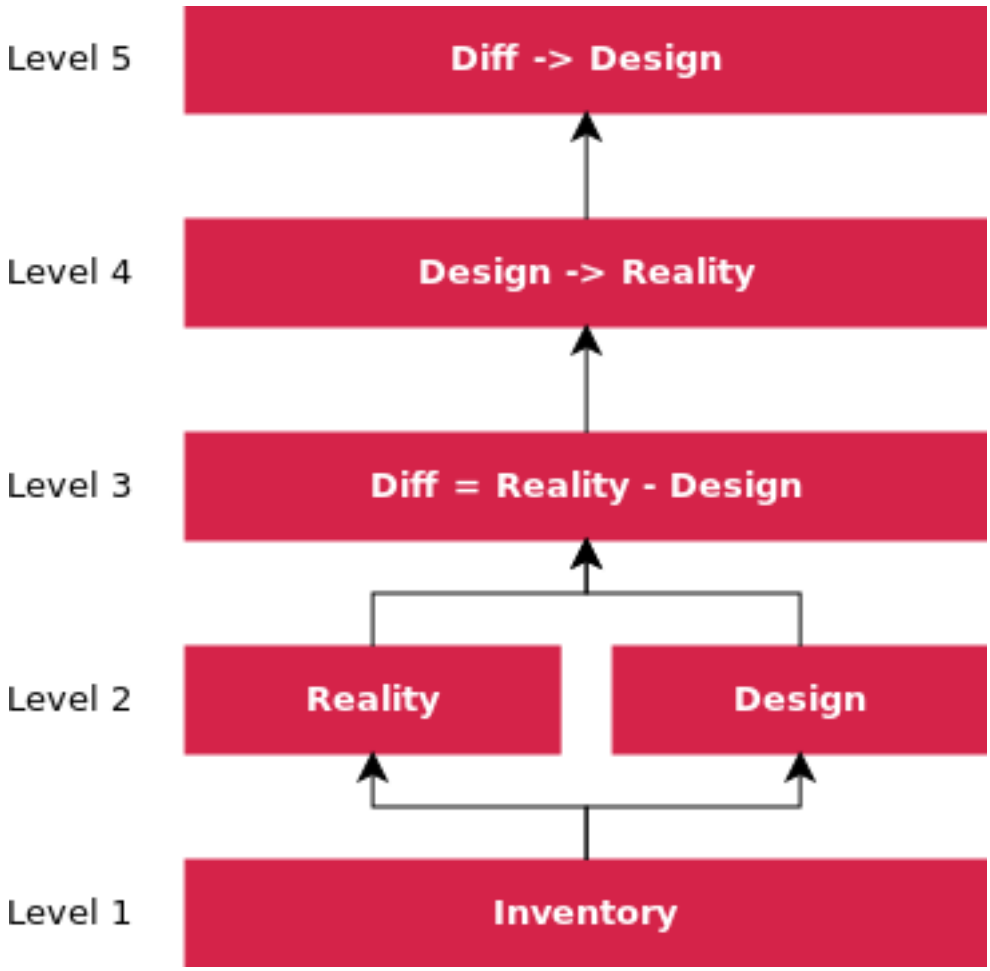

Figure 1: Five levels of capabilities

In this proposal, we will focus on the machinery of level 1 “inventory” and level 2 “reality/expectation”, which will lay the foundation work to the other three advanced levels.

The higher the maturity is, the more knowledge we are incorporating into this tool, which implies we first have such knowledge from practice, likely in an offiline format, then codify its execution, thus centralizing and fixating these knowledge.

4 function domains

Translating these framework into action, we can effectivly divide function into to these domains:

1. **inventory**: form/format to maintain managed subjects, be it a JSON, XML, Excel, or a DB.
2. **transportation**: SSH, telnet
3. **info query**: OS/BMC CLI
4. **parse info**: due to varieties of our HW, which I strongly believe this is the *norm, not exception*. Parsing data is an iterative process and can ben quite frustrating.
5. **save info to model**: once useful bits have been extracted by the parser, we will persist them to DB based on data models.
6. **report**: present data for our consumption, eg. policy offenders.
7. **apply action**: executing through a given interface to make a change on the target.

5 where we are today

5.1 inventory

There are two ways to inventory hardeware target: manually, and discovery.

- ☞ **manually**: giving a web form, eg. django admin
- ☞ **semi-automatic**: data import (excel). Initial data inputs are maintained outside Netbox.
- ☞ **discovery**: given minimal info and relies on the power of our knowledge of the hardware to enumerate and build up the inventory.

5.1.1 manually

1. working: Django admin is a given. But it's painfully slow.
2. working: Netbox have some form pages allowing user to fill in info.

5.1.2 semi-automatic

1. not working: Web interface has some kind of mass import. But this won't work w/ our customized data models.
2. TODO: customize mass import web page. This, however, also assumes the data format of imported data. Therefore, in case of Excel, it means we also dictates an Excel format, which makes knowledge residing in two places — in Netbox & in Excel, usually should be avoided.

DRAFT COPY

5.1.3 discovery

The idea is to start w/ a blank DB, and populate hardware inventory as much as possible using scripts. Tools such as confluent or my own script will do the job. It's a probe/READ-only action that will scan a list of given (ip, user, pwd), enumerate through some interfaces to try out, and see how much we can get by this info alone.

5.2 transportation

By which interface do we talk to a target? Fundamentally this is also a choice of tooling. Ansible, for example, implies SSH access. Decision here to have ripple effect on all the following steps that how info is queried, what is the data format, thus how we can parse it. Here we propose the principle of smallest common denominator, meaning:

1. **the lower level, the better**: get information at its source
2. **the commoner the presence, the better**: choose an interface giving us the most device coverage

Table 1: transportation

	host	BMC	TOR	Pica
transportation	SSH (linux only)	SSH	telnet	telnet

5.3 info query/parse/save

These three go hand-in-hand. As of writing 1/11/2021, we can achieve the followings:

Table 2: info query/parse/save

	host	BMC	TOR	Pica
info query	1. hostname	1. machine name	1. mac table	1. interface brief
parse info	2. interface list	2. eth0 mac	2. switch name	2. vlan interface brief

DRAFT COPY

	host	BMC	TOR	Pica
save info to DB	3. contents of any file	3. vpd sys	3. LLDP neighbors	3. vrrp settings
		4. power state	4. system info	4. running config
		5. firmware	5. mgmt port/mac	5. interface detail
		6. storage controller	6. port mac	6. (CP) contents of cldtx_switch_resources.sh
		7. storage drives	7. port enabled	7. (CP) contents of dhcpd.leases
		8. adapters	8. port is trunk	
		9. storage volumes	9. port native vlan	
		10. system boot mode	10. port allowed vlans	
			11. running config	

Of course, it depends on how the other side is configured, eg. whether it allows SSH connection at all, and what permission this `user` has. By default, we are expecting the user to be some kind of root access.

5.4 report

Netbox has a report module, which we haven't, however, used it. For our purpose, we are not seeking a general purpose reporting capability. Rather, we are interested in some report, for example, a report of hosts allowing the `(root, passw0rd)` access. Therefore, it will be more efficient to build such report as Django page.

Table 3: report

	host	BMC	TOR	Pica
report	1. device detail	1. device detail	1. device detail	1. device detail
	2. device list filtered	2. device list filtered	2. device list filtered	2. device list filtered

5.5 apply action

Netbox by design is a READ-only tool. It was not intended to apply change onto its inventory. Here we have differed completly from this principle because we believe this capability is a must-have if we are to enforce rules. Of course it is also convenient for batch operation, for example, shutting down entire lab in dawn of a planned power outage by enumerate BMCs and sets its power off.

Table 4: actions

	host	BMC	TOR	Pica
action	None	1. change USERID pwd w/ verification	1. change port vlan configs	1. set configuration
		2. set power state, eg. power on/off		2. set hostname

Alternatively, we have also considered Ansible, xClarity and confluent as action agent. We have had POC calling these tools from Netbox. Thus they are not excluded.

To use them, we must observe the principle that netbo is the sole source of truth. Therefore, netbox should dynamically generate inputs for them to consume on the fly, but will not permanently maintain these information outside netbox. For example, if to call Ansible for an action, IP list shoudl be generated by Netbox dynamically, but should not be a static file maintained separated from netbox, say, a confluence page. This is crucial also considering that actions can be taken parallely for scale purpose. Maintaining info outside netbox will introduce unnecessary complexities in

that regard.

6 TODOs

Each TODO below is a stream. It can be further broken down to smaller chunks. For now, we are estimating 2 man-week per stream.

6.1 policies

6.1.1 TODO rebase customization to upstream version

We have been 3 years behind upstream version. We should rebase our changes to the latest version.

1. establish two repos: one to track upstream, and one to track our changes.
2. port existing changes onto new upstream copy

6.1.2 TODO backup and restore TOR running configs

1. create a repo to track TOR configs
2. for each TOR:
    1. dump running config
    2. save config to repo
    3. establish cron routine to backup and commmit periodically

6.1.3 TODO achieve lab level isolation

Address the case when company IT shuts us down. We should have a single kill-switch, logically or physically, to disconnect our lab(s) from the campus. However broad the scope is, being it all the labs we own, or only US labs, or 2nd floor US lab, we need a single point of contact to turn on/off.

1. inventory rack-to-campus connections
2. inventory rack-to-rack connections
3. examine and redesign lab-to-campus topology: internal vlan naming
4. implement lab-to-campus topology: rewire, TOR config

DRAFT COPY

6.1.4 TODO achieve rack level isolation

Address the case when a rack went rougue. We should isolate rack from lab with a single contact.

1. redesign rack-to-rack topology
2. rewire and reconfig TOR for cross rack connections.

6.1.5 TODO switch user management

Switch is the key of our network security. We shouldn't allow root access to them unless we have tight monitoring or offline control.

1. create user account w/ less privilege
2. take away root pwd (only mgt knows) for *all*

6.1.6 TODO BMC user management

1. change USERID pwd periodically
2. create 2nd user for dev access

6.2 inventory

6.2.1 TODO unify switch name

Establish a switch naming convention, eg. R1U1, and make sure all switches are following this naming. This involves manually access switch serial port, if necessary, for CLI access.

6.2.2 TODO unify BMC name

Similar to the switch naming, establish a BMC naming convention and make all of them conform.

6.2.3 TODO re-examine hardware inventory, manually

Info may be scattered in Excel, netbox and others. Conslidate them into netbox. US locations only. Once we have achieve more robust discovery method, we will add Romania & India.

There will be four classes of devices from our point of view:

1. the ones we own and have full access: should gain full control
2. the ones we own, but we don't have full access: some team's dev rack that may or may not allow our access, eg. Openstack servers
3. the ones we don't own, and have full access: maybe they are using default root pwd?
4. the ones we don't own, and don't have full access: out of control

6.2.4 TODO new web form to relocate device easily

A form to log device moved from location A B. Right now, the Django admin interface is too slow, and it can only change one device at a time.

1. A data table of all devices (filtered) could be the way to go.
2. For BMC and TOR, name should be auto changed reflecting its new location.

6.2.5 TODO new web form to assign IP to a device

A batch way to assign IP to a list of devices. This is specially useful for BMC and TOR because they are always static.

6.2.6 TODO re-examine and re-org stack grouping

We have stacks belonging to differen projects, sometimes to different teams within a project. Netbox has some grouping notion, eg. site. These grouping value can be used for filtered list, and for reporting.

Let's establish a convention how we want to organize them.

1. by site: US, Romania, India
2. by tenants: CP, LOC, OSP. Within CP, cp-dev, cp-qa, cp-cicd, …
3. by rack

6.3 info query/parse/save

6.3.1 TODO verify physical connection algorithm

Tracking which wire is connected to which is impossible. However, before we can control who comes into the lab, it's impossible to exclude someone plugin a wire,

thus chaning the network topology both physically and logically.

We have developed some algorithms to derive physical connections. They need to be verified against a know connection. If we can achieve this, we are in a strong position to fully monitor and diagnose our lab.

6.3.2 TODO scale build inventory & meta from (ip,user,pwd)

Based on minimal set of info to build up the inventory and meta data in a full automated fashion. Presumbly location info can never be automated, but rest of info can.

As of writing, the challenge is network timeout — attempting an IP has at least 20s timeout. Therefore, scanning a 4096 IPs is a 22 hour run, too long to be practical.

1. interface to input list of (ip,user,pwd)
2. build enumeration tasks
3. use the Redis to scale out execution

6.3.3 TODO develop scan of storage controller

Add capability to scan CP storage controller.

6.3.4 TODO develop scan of Pica

Add more functions to Pica scan, for example, it's port config and connection.

6.3.5 TODO develop scan of DHCP service

Develop a way to scan and pinpoint DHCP service serving from our lab. In nearlly all cases, they are a sign of rule offender.

6.3.6 TODO research detection of VM vs. BM

I don't know whether this is possible. The idea is to be able to further categorize netbox devices into BM and VM, and if possible, establish logical relationship between them, eg. VM 123 is residing on BM R1U37. By WSS, this can be nested, thus only calling for a research.

Netbox has notion of cluster **for** VM and VM, they should be put to use.

1. interface to virsh
2. interface to CP: through CP, we can get VM list
3. interface to RHHI
4. interface to OSP

6.4 report

6.4.1 TODO build report given a IP range

By given an IP range, generate report of all devices known.

1. by CIDR
2. by a CSV list
3. by a range, eg. 10.240.31.1 - 10.240.45.2

ref: https://www.ipaddressguide.com/cidr

6.4.2 TODO build report of a stack

List all device and its details in a stack.

6.4.3 TODO build report of a grouping

1. by site
2. by tenant
3. by rack

6.4.4 TODO build report of physical connections

1. rack to rack
2. by rack
3. by tenant
4. by IP range

7 Opportunity

Now, an even bolder approach is to build a Lenovo's netbox from scratch with the same objective in mind. The advantage of Netbox was its data models. Without rein-

venting the wheels we have a blue print covering all aspects of a data center infrastructure. However, customization has also reveals its limitations.

Quote from its document:

> -IP address management (IPAM) - IP networks and addresses, VRFs, and VLANs -Equipment racks - Organized by group and site -Devices - Types of devices and where they are installed -Connections - Network, console, and power connections among devices -Virtualization - Virtual machines and clusters -Data circuits - Long-haul communications circuits and providers -Secrets - Encrypted storage of sensitive credentials

Out of these:

1. We are only using about 40% of its models.
2. Grouping by site and tenant feel limited, eg. is 2nd floor lab a site?
3. Expecting to log VMs manually is not practical.

On the other hand, we have extended functions that, I believe, are useful for any infra admin:

1. Scalability of long running tasks. Without this, scan and monitor will be impossible, thus greatly diminishing the value of it being the source of truth.
2. New model for chassis, controllers within chassis, and BMC controller. Even though we could use Device's parent-child relationship for this, it feels awkward. Nesting devices can further complicate query and reporting.
3. Algorithm to describe physical connection. This is one of a kind no other projects have ever achieved.

If considering the amount of work for new features listed above such as new page views, forms, which are conceptually straightforward, will be unnecessarily burdened by having to following the *Netbox way*. If we can develop a tool, similar to Netbox, by leverage some of its models and concepts, but focusing on some day-to-day issue our lab has, we may indeed find a tool that can help other lab managers in DCG also.

8 References